2022 International Conference on Machine Learning and Intelligent Systems Engineering (MLISE) | 978-1-6654-9246-1/22/$31.00 ©2022 IEEE | DOI: 10.1109/MLISE57402.2022.00011

Design of a YOLO Model Accelerator Based on PYNQ Architecture

Lin Wang[1], Tianyong Ao[*1], Le Fu[1], Jian Liu[2], Yang Liu[3,4], Yi Zhou[1,4]
[1]School of Artificial Intelligence, Henan University, Zhengzhou, China
[2]Guangdong Greater Bay Area Institute of Integrated Circuit and system, Guangzhou, China
[3]School of Computer and Information Engineering, Henan University, Kaifeng, China
[4]Shenzhen Research Institute of Henan University, Shenzhen, China
Corresponding author's e-mail: tyao@vip.henu.edu.cn

***Abstract*—The application requirements of object detection models based on deep learning are very extensive. However, high computing power requirements often seriously restrict the application of these models on resource-constrained devices with high energy efficiency requirements. To address this problem, a YOLO model accelerator architecture is proposed based on PYNQ. Based on the FPGA hardware platform, the hardware accelerator is designed by making full use of pipeline, loop unrolling, data reordering and other methods to accelerate the computationally intensive units in the YOLOv2 model such as the convolution and pooling layers. In order to reduce the delay in the data transmission process, the multi-channel transmission architecture combined with the ping-pong buffer is designed, and block-by-block reading strategy is adopted to read the off-chip data. The proposed YOLO model accelerator has been implemented and verified on Xilinx PYNQ-z2 platform. The experimental results show that the system has high detection accuracy and far lower power consumption than CPU and GPU. It can also be deployed on mobile devices to detect the surrounding environment.**

Keywords-FPGA; YOLO accelerator; object detection; low power consumption

I. Introduction

Artificial intelligence technology is widely used around us and has a profound impact on our daily life [1]. Object detection can be seen everywhere in the field of artificial intelligence, and the core of object detection is convolutional neural network (CNN)[2]. In the object detection network, the most representative algorithms include SSD, Faster R-CNN, YOLO. Compared with other models, the YOLO model performs fast and requires less computation. At the same time, the network maintains a better generalization ability, so it has high stability in new fields. Its idea is to transform the object detection problem into a regression problem. Because the full picture of the image is seen during the entire training and testing period, the prediction accuracy is greatly improved compared to other network models. Neural network usually uses CPU, GPU, ASIC and FPGA for inference acceleration. Compared to CPU and GPU, FPGA has low power consumption and high energy efficiency. Compared with ASIC, FPGA has high flexibility, low cost and short development cycle. At present, FPGA-based image recognition application scenarios are becoming more and more specific, such as the detection and tracking of UAVs, small medical assistance systems, lightweight robots, etc. On the premise of maintaining the detection accuracy, most of the detection methods focus on low power consumption and low cost. Therefore, there is a wide demand for object detection systems with high energy efficiency, low power consumption, and low cost. The research of FPGA accelerated object detection model is of great significance for the development of artificial intelligence[3].

There are plenty of methods to accelerate CNN based on FPGA. Nguyen et al binarized the weight and stored the entire network model in the memory block, which saved FPGA resources and reduced the number of accesses to the memory block, but the binarization of the weight had a great impact on the data accuracy[4]. Ma et al proposed to expand the output feature map, width and height, but due to the huge amount of data after expansion, it consumes too much FPGA resources[5]. In order to solve the delay problem of FPGA computing, J Zhang et al proposed a low delay accelerator architecture, which mainly reduced the inference calculation time of CNN by reducing the start-up time of pipeline[6]. However, in practical applications, data transmission between network layers is easily affected by memory delay. Zhang et al proposed a reconfigurable CNN accelerator based on the ARM+FPGA architecture[7]. The ARM processor completes the calculation in different inference processes by receiving the configuration signal information. Data quantization can reduce the storage space of parameters and computational complexity. However, the loss of precision caused by very low bits quantization is also very obvious. Therefore, how to balance the data accuracy and the consumption of FPGA resources is a key issue. In the design of the accelerator, the way of data transmission should be considered to improve the parallelism of computation and make full use of the advantages of FPGA parallel computation. Otherwise, the data transmission between the computing units and the storage system may become the bottleneck of the whole architecture.

At present, the YOLO model has been updated for five generations. The new version is difficult to be applied in the resource-constrained embedded field due to the large network model. In this paper, we propose a hardware accelerator to deploy YOLOv2 model on FPGA. The 16-bit fixed-point quantization is used to reduce the consumption of FPGA resources. In order to reduce the delay of data transmission, a method combining ping-pong buffer and multi-channel parallel computing is proposed.

The structure of the paper is organized as follows. In Section II, we analyze the structural features of the YOLOv2 model. Section III presents the architecture of the accelerator for YOLOv2 model and introduces the implementation of the accelerator in detail. In Section IV, the experimental results are analyzed. Finally, Section V concludes this paper.

978-1-6654-9246-1/22/$31.00 ©2022 IEEE
DOI 10.1109/MLISE57402.2022.00011

II. Analysis of YOLOv2 Model

The YOLOv2 model contains 23 convolution layers and 5 maximum pooling layers. The convolution layer is composed of convolution operations and activation functions, and the Leaky Relu function is used as the activation function. Each convolutional layer is designed with batch normalization to prevent the neural network from producing gradient vanishing during the inference calculation process. The YOLOv2 model uses a large number of 3×3 convolution kernels and 1×1 convolution kernels in the process of calculation. After the 3×3 convolution, a pooling operation is performed to extract the feature values, and the number of output channels is doubled at the same time. In order to keep the image size unchanged, the number of channels is reduced through a 1×1 convolution kernel.

After the neural network performs inference calculation, the calculation result is taken out from the network, including the coordinates, category and confidence information of the object. Then, the redundant boxes are filtered by non-maximum suppression to get the best detection result, and the detection result is fed back to the user.

III. Architecture of the YOLO Accelerator

A. Overall Architecture

The system adopts ARM+FPGA architecture to accelerate the inference of neural network, including processing system(PS) and programmable logic(PL). The PS realizes task scheduling and data distribution through the ARM processor, including memory control modules, data input and output interfaces, and classification results. The PL makes full use of the characteristics of FPGA that are good at parallel computing, and realizes complex computing of modules such as convolution and pooling. It is mainly composed of on-chip memory and multiple accelerated IP cores. The communication between PS and PL is completed through AXI bus. The architecture of the system is shown in Figure 1.

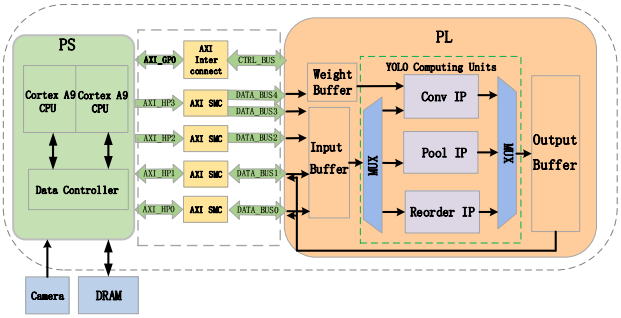


Figure 1. Design of overall architecture diagram

The conv IP mainly performs multiplication and addition operations through multiplication arrays and addition trees to improve the degree of parallelism of computation. The pool IP uses multiple comparators to calculate in parallel, finds the largest pixel value in the neighborhood, and writes it to the corresponding output buffer. The processing process of the reorder IP is similar to that of the pooling module. It samples the pixels of a single input feature map. We design a 4-way selector to read 1 pixel from the input buffer and write it to the corresponding output buffer each time.

The workflow of the system: firstly, the camera collects the external environment information, then the PS pre-loads the data. At the same time, the data is transferred to the hardware accelerator through DMAs. After the accelerator completes the computation, the result is returned to the PS. Finally, the image is classified on the PS side.

B. Data Transmission Architecture

In order to solve the problem of transmission delay in the data transmission process, we propose a multi-channel parallel data interaction method, and set up a ping-pong buffer to optimize the internal pipeline of the accelerator. We read off-chip feature map parameters in parallel using 4-channel DMA. Because in the process of the convolution operation, the size of the feature map parameters read each time just matches the size of the single-channel transmission weight parameters, so the weights are transmitted using a single channel.

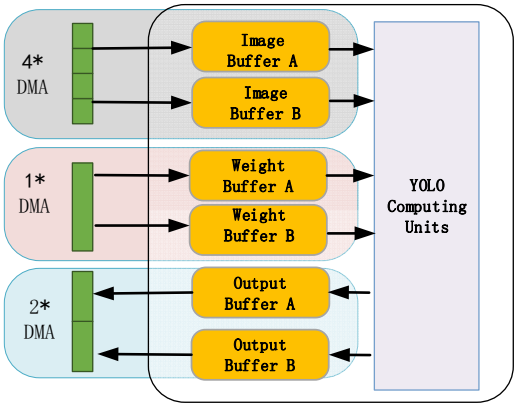


Figure 2. Data transmission architecture diagram

The entire computing process can be divided into three parts, namely reading data off-chip, processing data, and writing the result back to off-chip. Considering that these three parts can overlap during calculation, we design ping-pong buffers to implement pipeline operation. The schematic diagram is shown in Figure 2. After using the ping-pong buffers, the data is read from DRAM and stored in buffer A in the first clock cycle. In the second clock cycle, the data in the data buffer A is transmitted to the computing unit, and the feature map buffer B reads the data from the DRAM. At this time, the reading of the data and the calculation are performed at the same time. And so on, the data goes through all three processes in parallel. The strategy of multi-channel transmission combined with ping-pong buffers proposed reduces the inference time of hardware accelerators.

C. Design of Convolution Acceleration Module

Most of the computing time of the YOLO model is concentrated in the convolutional layers, and it is impossible to complete the operation of the entire convolutional layer on the FPGA at one time. In the convolution operation, the input image is a three-dimensional array, and performing the convolution operation according to the normal method will increase the calculation time. Therefore, we design multiple parallel

multiplication units and addition units, expand the input feature map according to the input channel and output channel, and use the block strategy to read data from off-chip to complete the convolution operation.

T_n and T_m indicates the number of parallel paths of input and output for each calculation, respectively. C_{in} and C_{out} indicates the number of channels in the input and output feature map, respectively. X_{in} and X_{out} indicates the width of the input and output feature map, respectively. y_{in} and y_{out} indicates the height of the input and output feature map, respectively. T_{xin} and T_{xout} indicates the width of the input and output feature map for each computation, respectively. T_{yin} and T_{yout} indicates the height of the input and output feature map for each computation, respectively. According to the block rule, the input channel is divided into C_{in}/T_n , and the output channel direction is divided into C_{out}/T_m . The size of the original input feature map on each channel is divided into$(X_{in}/T_{xin}) \times (Y_{in}/T_{yin})$, and the output feature map on the output channel is divided into $(X_{out}/T_{xout}) \times (Y_{out}/T_{yout})$. To sum up, the number of off-chip reading of convolution kernel for each level of convolution operation is $(C_{in}/T_n) \times (C_{out}/T_m) \times (X_{out}/T_{xout}) \times (Y_{out}/T_{yout})$, the number of off-chip reading of feature map is $(C_{in}/T_n) \times (X_{in}/T_{xin}) \times (Y_{in}/T_{yin})$, and the number of writing back from output cache to off-chip storage system is $(C_{out}/T_m) \times (X_{out}/T_{xout}) \times (Y_{out}/T_{yout})$.

The convolution layer has a large amount of computational operations. The data is divided into several small blocks by this method of block transmission, and the limited FPGA resources can be reused efficiently as much as possible to complete the operations.

D. Data Quantization

The neural network consumes a large number of multipliers and adders during the convolution operation, and the consumption of resources is closely related to the bit width of the data. Compared with 32-bit floating-point data, 16-bit fixed-point data will reduce the consumption of resources under the premise of preserving data precision. Compared with 32-bit floating-point data, 16-bit fixed-point data will reduce the consumption of resources, so we quantize 32-bit floating-point data into 16-bit fixed-point data for operation. The value of the quantized 16-bit fixed-point number can be represented by formula (1).

$$V_{fixed} = \sum_{i=0}^{15} B_i \times 2^{-fl} \times 2^i, \quad B_i \in \{0,1\} \tag{1}$$

B_i represents the binary number of the i-th digit, and fl represents the number of decimal places.

The weight size of the YOLOv2 model after training is 194MB, the data size after 16-bit fixed-point quantization is about 97MB, and the data volume is compressed by 50%. Therefore, the space opened up for the weight on the FPGA is 100MB and the consumption of DRAM resources is reduced by half . The resource consumption of the convolution after 16-bit fixed-point quantization is shown in Table 1.

Table 1. Resource consumption of convolution layer

Resource	DSP	FF	LUT
Number	130	11546	18987

IV. Experimental Results

A. Experimental Platform

The hardware architecture designed in this paper is evaluated on the Xilinx-PYNQ z2 board, which is equipped with a Cortex A9 chip, including programmable logic PL and embedded processor PS. The accelerator hardware core module is designed through Vivado HLS 2019.2, and use Vivado 2019.2 for synthesis and routing. The dataset is the COCO dataset.

B. FPGA Resource Consumption Assessment

The resource occupation of this design after placement and routing is shown in the following table 2. The table lists the usage of LUT, FF, BRAM, DSP and MMCM. We can find that LUT, BRAM, and DSP have high occupancy rates because each calculated data is stored in on-chip storage in the design. In convolution operation, the multiplication tree is used for a large number of multiplication operations, so DSP resource occupancy is high.

Table 2. FPGA resource consumption

Resource	**Utilization**	**Available**	**Utilization %**
LUT	26573	53200	50
FF	23794	106400	23
BRAM	88	140	63
DSP	151	220	69
MMCM	1	4	25

C. Analysis of Co-simulation Results

After the designed hardware accelerator is simulated and synthesized in Vivado 2019.2, the software and hardware co-simulation is performed. The waveform of reading and writing data when the system is running is shown in Figure 3.

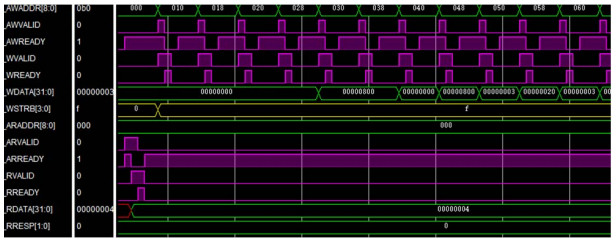


Figure 3. Waveform diagram of software and hardware co-simulation

The VALID and READY signals are preparation signals, and AXI has a handshake mechanism in the data transmission, as shown in the pink waveform in Figure 3. The WSTRB acts as an enabling signal, as shown by the yellow line in Figure 3. The write operation can run properly only when the signal is triggered during data transmission. As can be seen from the waveform diagram, the address is written first, and then the data is written after several clock cycles. After the write operation is complete, data is read. The waveform reflects the data interaction process during the normal operation of the system, and the working process is consistent with the expectation.

D. Comparison with Related Works

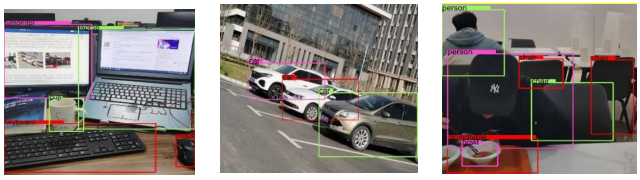
Table 3 compares this design with the previous YOLO hardware acceleration. In terms of power consumption, the power consumption of this design is only 1.81W, which is about 1/35 of the CPU and about 1/120 of the GPU. Compared with other types of FPGA, the power consumption is also the lowest, so our proposed design method reduces power consumption by a large margin.

Table 3. Comparison with related works

Model	Device	Frequency	Precision	Power(W)
YOLOv2 [8]	GTXtitanX	1GHz	float32	219
YOLOv2 [9]	Intel-i7670k	4000MHz	float32	65
Tincy-YOLO [10]	XCZU3EG	--	fixed8	6
TinyYOLOv3 [11]	Zedboard	100MHz	fixed16	3.36
This paper	Pynq-Z2	100Mz	fixed16	1.81

E. Detection Results in Actual Scenarios

To further evaluate the performance of the system, we mounted the camera module on the FPGA board to detect the surrounding environment in real time. Aiming at the stability of system detection, a large number of experiments have been carried out with this system in various scenarios, such as indoors, streets and restaurants. Some experimental results are shown in Figure 8. From the test results, we can see that all the information contained in the image can be accurately detected.



(a) Indoor (b) Street (c) Restaurant

Figure 4. Detection results in different scenarios

V. Conclusion

For the YOLO model, we have designed and implemented a hardware accelerator based on the PYNQ architecture. Firstly, we have explored the 16-bit fixed-point data quantization method, which greatly saves the consumption of hardware resources under the premise of ensuring data accuracy. Then, aiming at the computationally intensive convolution layer in the YOLO model, we have proposed a calculation method combining cyclic block and multiplicative array, which not only improves the computational efficiency, but also is suitable for convolution operations of different structure types. In order to reduce the data transmission delay, we have proposed a data transmission architecture using ping-pong buffer and multi DMA transmission. Finally, we have carried out a large number of experiments in different scenarios, and the results have shown that our design method not only reduces the system power consumption but also has a high recognition accuracy. It is of great significance for the object detection field and the acceleration of the hardware platform. In future research, we will further improve the versatility of the accelerator so that it can also be applied to other object detection models.

Acknowledgments

This work was supported from the Science and technology project of Henan Province(No. 212102210274) , and Shenzhen Special Foundation of Central Government to Guide Local Science & Technology Development(Nos. 2021Szvup029 and 2021Szvup032) , National Natural Science Foundation of China (No.62176087), Postgraduate Education Reform and Quality Province Improvement Project of Henan Province (No.YJS2022JC33), and Education Reform Research and Practice Project of Henan University (No.HDXJJG2020-109).

References

[1] Jiang, Yuchen, et al. "Quo vadis artificial intelligence?." *Discover Artificial Intelligence* 2.1 (2022): 1-19.

[2] Kaur, Davinder, et al. "Trustworthy artificial intelligence: a review." *ACM Computing Surveys (CSUR)* 55.2 (2022): 1-38.

[3] Yazdeen, Abdulmajeed Adil, et al. "FPGA implementations for data encryption and decryption via concurrent and parallel computation: A review." *Qubahan Academic Journal* 1.2 (2021): 8-16.

[4] Nguyen, Duy Thanh, et al. "A high-throughput and power-efficient FPGA implementation of YOLO CNN for object detection." *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 27.8 (2019): 1861-1873.

[5] Ma, Yufei, et al. "Optimizing the convolution operation to accelerate deep neural networks on FPGA." *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 26.7 (2018): 1354-1367.

[6] Zhang, Jinming, et al. "A low-latency fpga implementation for real-time object detection." *2021 IEEE International Symposium on Circuits and Systems (ISCAS)*. IEEE, 2021.

[7] Zhang, Shiguang, et al. "An fpga-based reconfigurable cnn accelerator for yolo." *2020 IEEE 3rd International Conference on Electronics Technology (ICET)*. IEEE, 2020.

[8] Li, Shuai, et al. "A novel FPGA accelerator design for real-time and ultra-low power deep convolutional neural networks compared with titan X GPU." IEEE Access 8 (2020): 105455-105471.

[9] Bi, Fanghong, and Jun Yang. "Target detection system design and FPGA implementation based on YOLO v2 algorithm." *2019 3rd International Conference on Imaging, Signal Processing and Communication (ICISPC)*. IEEE, 2019.

[10] Preußer, Thomas B., et al. "Inference of quantized neural networks on heterogeneous all-programmable devices." *2018 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2018.

[11] Yu, Zhewen, and Christos-Savvas Bouganis. "A parameterisable FPGA-tailored architecture for YOLOv3-tiny." International Symposium on Applied Reconfigurable Computing. Springer, Cham, 2020.